

Capstone Project: From AWS to Kubernetes

One connected, end-to-end DevOps task — every phase builds on the last

1. AWS

2. Linux

3. Git

4. Docker

5. Kubernetes

What you are building

You will provision a cloud server on **AWS**, configure it using **Linux** command-line skills, pull your application source code from a **Git**/GitHub repository onto that server, package the application into a **Docker** container image, and finally deploy that container onto a **Kubernetes** cluster running on the same server — exposing it to the internet through the server's public IP. Unlike five separate exercises, each phase below produces something the next phase directly depends on: the EC2 instance you launch in Phase 1 is the exact machine you configure in Phase 2, clone your repo onto in Phase 3, containerize on in Phase 4, and run Kubernetes on in Phase 5.

Architecture at a glance

```
[AWS]          [Linux]          [Git]          [Docker]          [Kubernetes]
EC2 instance  --> configured with --> app source code --> packaged as an --> deployed as a
+ security group      Docker, Git,      cloned from      image and run      Pod behind a
+ key pair            kubectll, Minikube  the EC2 host      as a container    Service, exposed
                                                                        locally first     on a NodePort
```

Learning objectives

- Provision and secure a cloud virtual machine on AWS
- Administer that Linux server: users, permissions, installing software
- Manage application source code with Git and pull it from GitHub
- Containerize the application with a Dockerfile and Docker Compose
- Deploy and expose the containerized app on a Kubernetes cluster

Project details

Total duration:	10–15 hours across 5 phases (can be split over multiple sessions)
Starting point:	A free-tier AWS account + a free GitHub account + AWS CLI installed on your own laptop
Ending point:	A containerized web app, defined as code in Git, running on Kubernetes, reachable at <code>http://<EC2-public-ip>:30080</code>
Submission:	One report containing the command output / screenshots requested at the end of each phase, plus the GitHub repo link

Rules

- Do the phases **in order** — Phase 2 assumes the EC2 instance from Phase 1 is running; Phase 5 assumes Docker from Phase 4 is installed.
- Use only AWS Free Tier resources (`t2.micro` / `t3.micro`). Terminate everything at the end (Phase 5 clean-up).

- Never commit AWS keys, `.pem` files, or passwords to Git.

1 Phase 1 — AWS: Provision the Server

You will produce: A running EC2 instance with a public IP, a security group, and an SSH key — this is the machine every later phase uses

Run these commands: On your own laptop, using the AWS CLI

Step 1: Configure the AWS CLI

```
aws --version
aws configure
# AWS Access Key ID: <paste yours>
# AWS Secret Access Key: <paste yours>
# Default region name: us-east-1
# Default output format: json

aws sts get-caller-identity
```

Step 2: Create an SSH key pair

```
aws ec2 create-key-pair --key-name capstone-key --query 'KeyMaterial' --output text > capstone-key.pem
chmod 400 capstone-key.pem
```

Step 3: Create a security group

Open SSH (22) for you to connect, HTTP (80) for the web server test in Phase 2, and 30080 for the Kubernetes NodePort you will use in Phase 5 — this way you never have to come back and edit it later.

```
aws ec2 create-security-group --group-name capstone-sg --description "Capstone project SG"

aws ec2 authorize-security-group-ingress --group-name capstone-sg --protocol tcp --port 22 --cidr
0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-name capstone-sg --protocol tcp --port 80 --cidr
0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-name capstone-sg --protocol tcp --port 30080 --cidr
0.0.0.0/0
```

Step 4: Launch the EC2 instance

```
aws ec2 run-instances \
  --image-id ami-0c101f26f147fa7fd \
  --instance-type t2.micro \
  --key-name capstone-key \
  --security-groups capstone-sg \
  --count 1 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=capstone-server}]'
```

The AMI ID changes by region/time — look up the current Amazon Linux 2023 Free-Tier AMI ID for your region in the EC2 console before running this. Amazon Linux is assumed for the yum commands used later in this project.

Step 5: Get the public IP and connect

```
aws ec2 describe-instances --filters "Name=tag:Name,Values=capstone-server" \
  --query "Reservations[].Instances[].PublicIpAddress" --output text
```

```
ssh -i capstone-key.pem ec2-user@<public-ip>
```

Hand-off to Phase 2: Keep this SSH session open (or reconnect with the same command) — every command from here through the end of Phase 5 runs **inside this EC2 instance**, not on your laptop.

Checkpoint — Phase 1

- ☐ Output of `aws sts get-caller-identity`
- ☐ Output of `aws ec2 describe-instances ...` showing the public IP
- ☐ Screenshot of a successful `ssh` login prompt

2 Phase 2 — Linux: Configure the Server

You will produce: A configured EC2 host with a dedicated app user, correct permissions, and Git/Docker installed and ready

Run these commands: Inside the SSH session on the EC2 instance from Phase 1

Step 1: Update the system and check resources

```
sudo yum update -y
df -h
free -m
uptime
```

Step 2: Create a dedicated application user

```
sudo useradd -m -s /bin/bash appuser
sudo usermod -aG wheel appuser    # gives appuser sudo rights on Amazon Linux
sudo passwd appuser
id appuser
```

Step 3: Create the project directory and set permissions

```
sudo mkdir -p /opt/capstone-app
sudo chown appuser:appuser /opt/capstone-app
sudo chmod 750 /opt/capstone-app
ls -ld /opt/capstone-app
```

Step 4: Switch to the app user for the rest of the project

```
su - appuser
cd /opt/capstone-app
pwd
```

Step 5: Install Git and Docker

```
sudo yum install -y git
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker appuser
# log out and back in (exit the su session, then su - appuser again) for the docker group to apply
exit
su - appuser
docker ps
git --version
```

Step 6: Write a quick health-check script

```
cat > /opt/capstone-app/healthcheck.sh << 'EOF'
#!/bin/bash
echo "=== Server Health: $(date) ==="
df -h / | tail -1
free -m | grep Mem
docker ps --format "table {{.Names}}\t{{.Status}}"
```

```
EOF
```

```
chmod +x /opt/capstone-app/healthcheck.sh  
/opt/capstone-app/healthcheck.sh
```

Hand-off to Phase 3: You are now logged in as appuser inside /opt/capstone-app, with Git and Docker ready. The next phase clones your application code into this exact folder.

Checkpoint — Phase 2

- ☐ Output of `id appuser` and `ls -ld /opt/capstone-app`
- ☐ Output of `docker ps` and `git --version` run as `appuser`
- ☐ Output of running `healthcheck.sh`

3 Phase 3 — Git: Bring Your Code to the Server

You will produce: A GitHub repository containing a small Node.js app, cloned onto the EC2 instance at `/opt/capstone-app`

Run these commands: Steps 1–3 on your laptop; Step 4 onward back inside the EC2 SSH session as `appuser`

Step 1: Create the app locally and initialize Git

```
mkdir ~/capstone-app && cd ~/capstone-app
git init
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

cat > app.js << 'EOF'
const http = require('http');
const server = http.createServer((req, res) => {
  res.end('Hello from the AWS-to-Kubernetes capstone project!\n');
});
server.listen(3000, () => console.log('Server running on port 3000'));
EOF

echo "node_modules/" > .gitignore
echo "*.pem" >> .gitignore
```

Step 2: Commit

```
git add .
git commit -m "Initial commit: basic Node.js app"
git log --oneline
```

Step 3: Push to GitHub

Create a new empty repository named `capstone-app` on github.com (do not add a README there), then:

```
git remote add origin https://github.com/<your-username>/capstone-app.git
git branch -M main
git push -u origin main
```

Step 4: Clone it onto the EC2 server

```
# back inside the SSH session, logged in as appuser, in /opt/capstone-app
git clone https://github.com/<your-username>/capstone-app.git .
ls -la
cat app.js
```

Step 5: Practice a real change-and-pull cycle

```
# on your laptop:
cd ~/capstone-app
git checkout -b feature/update-message
sed -i "s/capstone project!/capstone project, deployed with Docker + K8s!/" app.js
git add app.js
git commit -m "Update greeting message"
git push -u origin feature/update-message
```

```
# merge this branch into main on GitHub (Pull Request), then:
git checkout main
git pull origin main

# on the EC2 server:
cd /opt/capstone-app
git pull origin main
cat app.js
```

Hand-off to Phase 4: /opt/capstone-app on the EC2 server now holds your latest app.js. The next phase turns this exact folder into a Docker image.

Checkpoint — Phase 3

- ☐ Link to your GitHub repository
- ☐ Output of `git log --oneline` showing at least 2 commits
- ☐ Output of `cat app.js` on the EC2 server showing the updated message

4 Phase 4 — Docker: Containerize the App

You will produce: A Docker image of your app, tested as a running container on the EC2 server, and pushed to Docker Hub so Kubernetes can pull it in Phase 5

Run these commands: Inside the EC2 SSH session, in `/opt/capstone-app`, logged in as `appuser`

Step 1: Write the Dockerfile

```
cat > Dockerfile << 'EOF'
FROM node:20-alpine
WORKDIR /app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
EOF
```

Step 2: Build the image

```
docker build -t capstone-app:1.0 .
docker images
```

Step 3: Run and test the container locally on the server

```
docker run -d --name capstone-container -p 3000:3000 capstone-app:1.0
docker ps
curl http://localhost:3000
docker logs capstone-container
docker stop capstone-container && docker rm capstone-container
```

Step 4: Push the image to Docker Hub

Kubernetes (Phase 5) needs to pull this image from a registry, so push it to Docker Hub (create a free account at hub.docker.com if you don't have one):

```
docker login
docker tag capstone-app:1.0 <your-dockerhub-username>/capstone-app:1.0
docker push <your-dockerhub-username>/capstone-app:1.0
```

Step 5 (optional but recommended): Add Docker Compose

```
cat > docker-compose.yml << 'EOF'
version: "3.9"
services:
  web:
    image: <your-dockerhub-username>/capstone-app:1.0
    ports:
      - "3000:3000"
EOF
docker compose up -d
curl http://localhost:3000
docker compose down
```

Hand-off to Phase 5: Your image <your-dockerhub-username>/capstone-app:1.0 is now on Docker Hub. The next phase tells Kubernetes to pull and run exactly this image.

Checkpoint — Phase 4

- ☐ Your `Dockerfile`
- ☐ Output of `docker images` and `curl http://localhost:3000`
- ☐ Link to your image on Docker Hub (hub.docker.com/r/<username>/capstone-app)

5 Phase 5 — Kubernetes: Deploy & Expose the App

You will produce: The app from Phase 4, running as a Kubernetes Deployment, exposed via a Service on the EC2 server's public IP

Run these commands: Inside the EC2 SSH session, logged in as appuser (Minikube install needs sudo)

Step 1: Install kubectl and Minikube

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

kubectl version --client
minikube version
```

Step 2: Start the cluster (using the Docker installed in Phase 2/4)

```
minikube start --driver=docker --force
kubectl cluster-info
kubectl get nodes
```

Step 3: Write the Deployment manifest for your image

```
cd /opt/capstone-app
cat > deployment.yaml << 'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: capstone-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: capstone
  template:
    metadata:
      labels:
        app: capstone
    spec:
      containers:
        - name: capstone-app
          image: <your-dockerhub-username>/capstone-app:1.0
          ports:
            - containerPort: 3000
EOF
```

Step 4: Apply the Deployment

```
kubectl apply -f deployment.yaml
kubectl get deployments
```

```
kubectl get pods -o wide
kubectl describe deployment capstone-deployment
```

Step 5: Write and apply the Service manifest

```
cat > service.yaml << 'EOF'
apiVersion: v1
kind: Service
metadata:
  name: capstone-service
spec:
  type: NodePort
  selector:
    app: capstone
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30080
EOF

kubectl apply -f service.yaml
kubectl get services
```

Step 6: Access the app from outside AWS

Because Minikube runs its own internal network, tunnel/forward the NodePort so it is reachable on the EC2 host's own IP, then test it from your laptop:

```
# inside the EC2 SSH session:
kubectl port-forward --address 0.0.0.0 service/capstone-service 30080:3000 &

# from your laptop:
curl http://<ec2-public-ip>:30080
```

Port 30080 is already open in the security group you created in Phase 1 — this is the pay-off of planning the security group ahead of time.

Step 7: Prove resilience — scale and roll out an update

```
kubectl scale deployment capstone-deployment --replicas=5
kubectl get pods

kubectl set image deployment/capstone-deployment capstone-app=<your-dockerhub-username>/capstone-app:1.0
kubectl rollout status deployment/capstone-deployment
kubectl rollout history deployment/capstone-deployment
```

Step 8: Final clean-up (stop AWS charges)

```
# inside EC2:
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
minikube stop
exit          # leave appuser shell, back to ec2-user
exit          # close SSH session

# on your laptop:
aws ec2 terminate-instances --instance-ids <your-instance-id>
```

```
aws ec2 delete-security-group --group-name capstone-sg
aws ec2 delete-key-pair --key-name capstone-key
```

Confirm termination with `aws ec2 describe-instances` before you close the lab — a forgotten running instance is the most common source of unexpected AWS charges.

Checkpoint — Phase 5 (Final Submission)

- ☐ Your `deployment.yaml` and `service.yaml`
- ☐ Output of `kubectl get pods -o wide` showing 3+ running Pods
- ☐ Screenshot of `curl http://<ec2-public-ip>:30080` returning your app's message, run from your own laptop
- ☐ Output of `kubectl rollout history deployment/capstone-deployment`
- ☐ Confirmation screenshot that the EC2 instance was terminated

Bonus Challenge — Full Loop

Change the greeting message in `app.js`, commit and push it to GitHub (Phase 3 skills), rebuild and push a new Docker image tagged `:2.0` (Phase 4 skills), then update the Kubernetes Deployment to the new image and watch `kubectl rollout status` perform a zero-downtime rolling update (Phase 5 skills) — proving the whole pipeline works end to end.